

AI 创作平台

项目经验讲解版 / 面试复盘材料

基于 Vue 3、FastAPI、LangGraph、SSE 和 RAG 的多 Agent 智能写作系统

项目定位	面向小说/长文创作者的 AI 辅助创作平台，支持素材管理、章节编辑、多 Agent 生成、润色、续写、读者反馈。
推荐角色描述	全栈开发 / AI 应用开发：负责前后端核心链路、LangGraph 工作流、SSE 流式生成、RAG 上下文注入和 LLM 配置。
技术栈	Vue 3 + TypeScript + Pinia + Vue Router；FastAPI + SQLAlchemy Async + PostgreSQL；LangGraph；JWT；Docker Compose。
核心亮点	多 Agent 编排、RAG 双路径上下文、SSE token 级输出、人机协同审批、多厂商 LLM 适配、API Key 加密存储。
生成日期	2026-05-22

LangGraph

RAG

SSE

FastAPI

Vue 3

PostgreSQL

一、30 秒讲清楚这个项目

这是一个 AI

小说/长文创作平台。用户可以创建创作项目，维护章节、大纲、角色、世界观和暗线等素材；写作时可以让 AI 以多个 Agent 的形式协作，例如先加载上下文，再由写作 Agent 生成内容，同时由审校 Agent 和一致性检查 Agent 给出反馈，最后交给用户确认是否写入章节。

项目的重点不是简单调用一次大模型，而是把 AI 写作拆成可编排、可观察、可人工介入的工程流程：前端用 SSE 实时展示生成过程，后端用 LangGraph 管理多节点状态流转，用 RAG 把项目素材注入 prompt，并支持用户按项目配置不同的大模型。

可以放到简历里的版本

- 设计并实现 AI 创作平台，支持项目、章节、角色、世界观、大纲、暗线等创作素材管理。
- 基于 LangGraph 构建多 Agent 写作流水线，实现上下文加载、章节生成、审校、一致性检查和人工审批。
- 通过 SSE 实现 token 级流式输出，前端可实时展示 AI 生成过程，并按生成/润色/续写模式差异化处理结果。
- 实现 RAG 上下文注入：用户主动选择素材时按 ID 精确加载，否则基于 embedding 做相关素材检索。
- 抽象 LLM Provider，兼容 OpenAI 协议模型；支持用户级 LLM 配置和 API Key 加密存储。

两分钟展开讲法

我把系统分成四层：前端工作台、后端业务 API、AI 工作流层和数据/RAG 层。前端负责编辑器、素材选择和 Agent 面板；后端负责认证、CRUD、SSE 输出和生成流程分流；AI 层用 LangGraph 表达多 Agent 协作；数据层用 PostgreSQL 保存项目素材、用户配置和章节内容。

其中最值得讲的是生成链路。用户点击“章节生成”后，会先选择大纲、角色、世界观和目标字数。后端拿到这些选择后构造 CreativeState，context_loader 节点加载素材，writer 节点生成小说正文，critic 与 consistency_checker 并行做审校和一致性检查。最后 human_review 暂停，让用户接受、修改后接受或拒绝。

二、系统架构

层级	主要模块	职责
前端表现层	Vue 3、TypeScript、Pinia、Vue Router	登录、项目列表、写作工作台、素材管理、Agent 配置、LLM 设置。
通信层	REST API + SSE	普通 CRUD 走 REST；AI 生成走 SSE，按事件类型推送进度、token、审校意见和错误。
业务 API 层	FastAPI、Pydantic、SQLAlchemy Async	认证、权限校验、项目归属校验、参数校验、数据库读写、导出。
AI 编排层	LangGraph StateGraph	把上下文加载、写作、审校、一致性检查和人工审批组织为状态图。
RAG/模型层	Embedding、ContextLoader、LLM Provider	加载创作素材，生成上下文 prompt，调用 mock 或 OpenAI 兼容模型。
数据层	PostgreSQL、Alembic、Docker Compose	保存用户、项目、章节、角色、世界观、大纲、暗线、专家和 LLM 配置。

核心目录说明

backend/main.py	FastAPI 应用入口，挂载 CORS、中间件和路由，启动时初始化数据库。
backend/api/routes.py	核心业务路由：项目/章节/素材/专家 CRUD、章节生成、 workflow 恢复、导出。
backend/agents/workflow.py	LangGraph 工作流定义，包含 CreativeState、默认节点和动态图构建。
backend/agents/llm_provider.py	LLM 抽象层，MockProvider 与 OpenAIProvider 统一 generate / generate_stream 接口。
backend/rag/context_loader.py	上下文加载和向量相似度检索。
frontend/src/views/Workspace.vue	主工作台，整合编辑器、章节列表、素材面板和 Agent 面板。
frontend/src/components/AgentPanel.vue	生成入口和 SSE 事件处理核心。
frontend/src/api/clients	REST 与 SSE 请求封装，包含鉴权 header 和错误解析。

数据模型

模型	用途	关键字段
Project	创作项目	title、description、target_words、mode、owner_id
Chapter	章节	project_id、title、content、outline、sequence_number、word_count、status

模型	用途	关键字段
WorldEntry	世界观条目	category、title、content、rules、confidence、embedding
Character	角色	name、role_type、profile、faction、metadata、embedding
Outline	大纲	sequence_number、title、summary、turning_point、hidden_thread_ids
HiddenThread	暗线/伏笔	name、description、chapter_nums
Expert	Agent 配置	role_type、system_prompt、workflow_position、temperature、max_tokens
LLMConfig	用户模型配置	provider、encrypted_api_key、base_url、model_id

三、核心生成流程

章节生成: full_pipeline

- 前端打开 ContextPicker，让用户选择大纲、角色、世界观，并填写目标字数。
- 前端调用 POST /api/projects/{project_id}/chapters/generate, mode=full_pipeline，并建立 SSE 连接。
- 后端校验 JWT、项目归属、章节是否存在，再读取用户 LLM 配置。
- 查询项目启用的 Expert，动态构建 LangGraph 工作流。
- context_loader 加载上下文：有选中素材就按 ID 精确加载；没有选中素材则走 RAG 检索。
- writer 生成章节草稿；critic 和 consistency_checker 分别审校与一致性检查。
- 前端接收 writer_output，把纯小说内容放进 finalDraft；审校和一致性报告只进入输出日志。
- done 后前端弹出 ApprovalModal，用户接受后写入章节编辑器并保存。

润色: enhance

润色被设计成两步：第一次请求不传 enhance_direction，后端只分析当前章节并返回 3 个润色方向；用户选择方向并补充要求后，第二次请求才真正改写全文。这样避免 AI 盲目润色，也让用户对改写目标有控制权。

续写: continue

续写同样是两步：先返回 5 个情节转折建议，再按用户选中的方向续写。与润色不同，续写 token 会直接追加到编辑器当前正文末尾，所以用户看到的是实时接龙式创作。

读者视角: summarize

summarize

模式不改正文，只让模型以普通读者身份评价章节的吸引力、节奏、角色立体感和情节合理性。它本质上是反馈型 Agent。

SSE 事件设计

事件	含义	前端处理
progress	系统进度	追加到输出区。
agent_start / agent_done	节点开始/结束	更新工作流状态。
writer_output	小说正文 token 或完整内容	按模式写入 finalDraft 或直接追加编辑器。
critic_output	审校意见	展示在输出日志，不污染正文。
consistency_check	一致性报告	展示在输出日志，不污染正文。
enhance_directions	润色方向	弹出 EnhancePicker。
turn_suggestions	续写方向	弹出 TurnPicker。
done / error	完成或失败	停止生成状态，显示审批或错误提示。

四、技术亮点怎么讲

1. 为什么用 LangGraph

这个项目的 AI 流程不是一次模型调用，而是一个多步骤工作流。LangGraph 适合表达这种“先加载上下文、再写作、再并行审校、最后人工审批”的状态图。每个节点只读写 CreativeState 里自己负责的字段，流程可视化、可暂停，也方便之后插入自定义 Agent。

2. 并行审校的价值

writer 产出草稿后，critic 和 consistency_checker 都只依赖草稿，不互相依赖，所以可以并行执行。串行时总耗时约等于两个模型调用相加，并行后约等于较慢的那个调用，理论上能明显降低等待时间。

3. RAG 双路径上下文

用户主动选择素材时，系统按 ID 精确加载完整大纲、角色和世界观，确保用户选择的内容不丢；用户没有选择时，系统基于当前章节内容做 embedding 检索，自动找相关角色和世界观。两条路径互斥，避免重复和上下文污染。

当前实现边界

仓库使用 pgvector 镜像作为数据库环境，但 MVP 代码里 embedding 字段是 Float 数组，检索是在 Python 内存中计算余弦相似度。这个设计适合小规模项目素材，后续可迁移到 pgvector 向量列和数据库索引。

4. 流式输出与正文纯净性

生成流程会产生正文、日志、审校意见、一致性报告等多种信息。前端把 writer_output 和其他事件严格分开：正文进入 finalDraft 或编辑器，审校与报告只进入 streamOutput。这样审批弹窗和最终保存的内容始终是纯小说正文。

5. 多厂商 LLM 适配

LLMProvider 定义 generate 和 generate_stream 两个接口。MockProvider 便于无 Key 开发测试；OpenAIProvider 使用 OpenAI 兼容协议，DeepSeek、通义千问、智谱、Moonshot 等只需要配置不同 base_url 和 model。用户 API Key 用 Fernet 加密后存数据库。

五、前端实现重点

工作台组织

Workspace.vue 是主工作区，进入项目后会加载章节、世界观、角色、角色关系、大纲和暗线。NovelEditor 负责正文编辑和自动保存，AgentPanel 负责 AI 生成入口与 SSE 事件消费。

状态管理

useProjectStore	项目列表、当前项目、创建项目。
useChapterStore	章节列表、当前章节、草稿更新、保存。
useExpertStore	Agent 列表、生成状态、流式日志、finalDraft、工作流步骤。
useWorldEntryStore / useCharacterStore / useOutlineStore	素材 CRUD 与按项目分组。
useLLMSettingsStore	加载/保存 LLM 配置、获取模型列表。
useUiStore	Toast 通知。

三种生成模式的前端差异

模式	用户交互	writer_output 去向	完成后动作
full_pipeline	选择上下文和目标字数	累积到 finalDraft	弹出审批，接受后写入章节。
enhance	先选润色方向	累积到 finalDraft	完成后替换当前章节内容并保存。
continue	先选转折方向	直接追加到章节 draft	实时写入编辑器，无需额外应用。
summarize	一键读者反馈	展示为反馈文本	不修改正文。

六、后端实现重点

认证与权限

- 注册时用 bcrypt 哈希密码；登录后签发 JWT，前端保存在 localStorage。
- 所有项目资源请求都通过 get_current_user 校验登录状态。
- 访问项目内资源前调用 _verify_project_owner，确保用户只能操作自己的项目。
- 生成和认证相关接口有简单限流器，降低误触或滥用带来的成本风险。

数据一致性

- 创建项目后自动写入 6 个内置 Expert 模板，保证新项目开箱即用。
- 创建章节时校验同一项目内 sequence_number 不重复。

-
- 世界观和角色变更后可后台更新 embedding，失败不阻塞主流程。
 - 导出接口按章节 sequence_number 排序输出 txt 或 md。

七、面试高频问答

问题	回答要点
Q1: 项目最大难点是什么?	不是调通模型，而是把生成过程工程化：要区分正文与审校日志，要支持流式输出，要能让用户控制上下文和生成方向，还要保证用户资源隔离。
Q2: 为什么不用普通串行函数?	串行函数可以跑 MVP，但难以表达并行审校、人工暂停和动态插入 Agent。LangGraph 把流程状态显式化，更适合扩展。
Q3: RAG 怎么保证不乱引用?	用户选择素材时不走召回，直接按 ID 加载完整内容；只有没有选择时才做相似度检索。主动选择优先级最高。
Q4: SSE 断开怎么办?	前端用 AbortController 取消请求，已写入编辑器的 token 保留；当前实现没有完整的跨连接重放，后续可用持久化 checkpoint 和事件日志增强。
Q5: API Key 怎么保护?	后端只保存 Fernet 加密后的密文，对外只返回 api_key_set 布尔值；生产环境还应把密钥管理迁移到 KMS。
Q6: 项目有什么可改进?	把 MemorySaver 换成数据库 checkpoint；RAG 从 Python 内存相似度升级到 pgvector 索引；加入 reranker；完善 E2E 测试和生成任务队列。

八、诚实讲项目边界

- 当前 LLM 默认 mock，因此无需 API Key 可以演示流程；真实模型需要在设置页配置 provider、base_url、model 和 API Key。
- LangGraph checkpoint 当前使用 MemorySaver，服务重启后暂停状态会丢；代码里预留了数据库 checkpoint 开关，但当前 get_creative_app 仍是内存实现。
- RAG 在小规模素材下用 Python 余弦相似度足够，生产大规模场景应改为 pgvector 索引、混合检索和重排序。
- 前端 store 初始化保留 mock 数据作为兜底，后端不可用时仍能预览 UI；正式环境应明确关闭或隔离 mock fallback。

启动方式

```
docker compose up -d
cd backend && pip install -r requirements.txt && uvicorn main:app --reload
cd frontend && npm install && npm run dev
```

一句话收尾

这个项目可以重点体现：我能把大模型能力落到真实产品流程里，处理上下文、流式交互、状态编排、用户配置、安全存储和前后端工程化，而不是停留在简单 prompt 调用。